

CloudAppDev

Milestone 1: Cloud-Native Travel Platform

IaaS vs. PaaS auf Google Cloud Platform

Präsentation starten

Travel Itinerary Management Platform

Kernfunktionen

- Reiserouten-Management (CRUD)
- User Authentication
- Image Upload zu Cloud Storage
- Comments & Likes System

Tech Stack

- Next.js 15 + React 19
- PostgreSQL + MongoDB/Firestore
- Firebase Authentication
- Google Cloud Platform



Tech Stack

Frontend

```
// Next.js 15.5.4 + React 19
import { PrimeReact } from 'primereact'
import { Tailwind } from '@tailwindcss'
```

- PrimeReact UI Components
- Tailwind CSS

Backend

```
// Next.js API Routes
import { PrismaClient } from '@prisma/client'
import { MongoClient } from 'mongodb'
import { Storage } from '@google-cloud/storage'
import admin from 'firebase-admin'
```

- Prisma ORM
- MongoDB Driver
- Firebase Admin SDK
- Cloud Storage SDK

Multi-Database Strategie

PostgreSQL 16

```
model User {  
  id          Int          @id  
  email       String       @unique  
  googleUid   String?      @unique  
  itineraries Itinerary[]  
  comments    Comment[]  
}
```

```
model Itinerary {  
  id          Int          @id  
  user_id     Int  
  title       String  
  locations   Location[]  
}
```

```
model Location {  
  id          Int          @id  
  itinerary_id Int  
  name        String  
  images      String[]  
}
```

MongoDB / Firestore

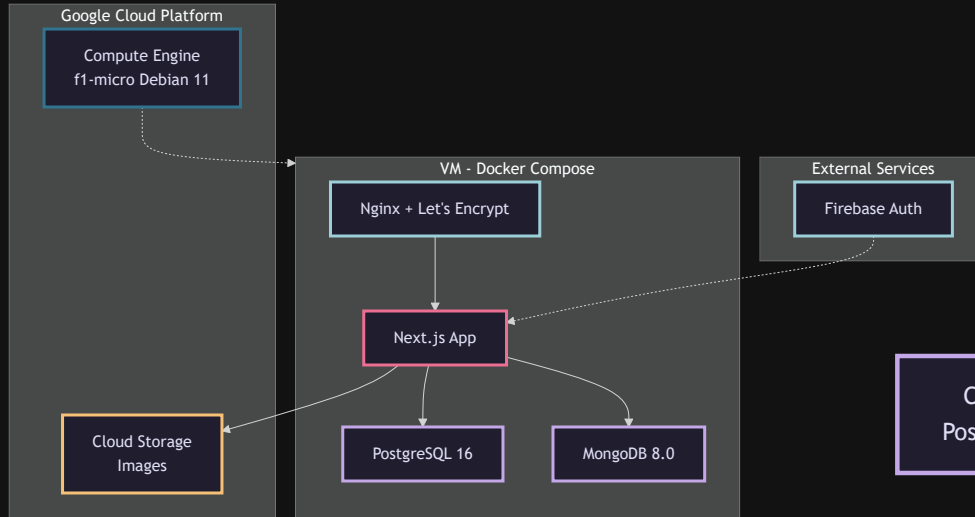
```
// Likes Collection  
{  
  _id: ObjectId,  
  itineraryId: 123,  
  userId: 456,  
  createdAt: ISODate  
}  
  
// Comments Collection  
{  
  _id: ObjectId,  
  itineraryId: 123,  
  userId: 456,  
  text: "Great route!",  
  createdAt: ISODate  
}
```

IaaS vs. PaaS Deployment

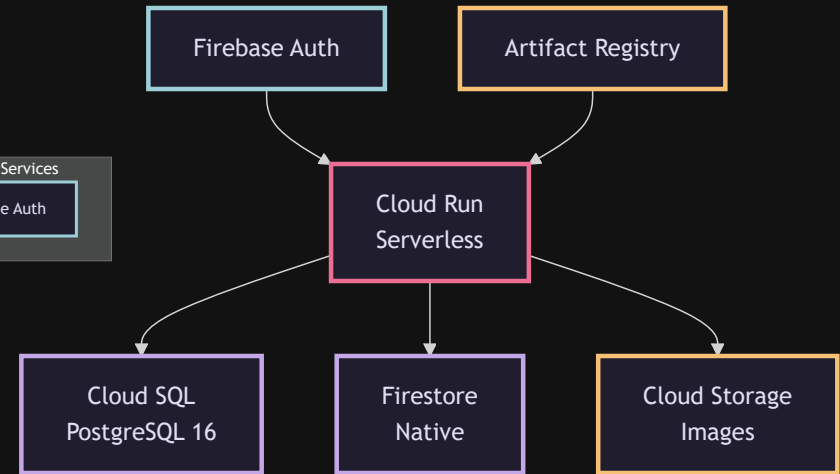
Zwei vollständige Implementierungen auf Google Cloud Platform

Architektur-Vergleich

IaaS - Infrastructure as a Service



PaaS - Platform as a Service



IaaS Implementation

Terraform Infrastructure

```
resource "google_compute_instance" "vm" {
  name          = "cloudappdev-iaas"
  machine_type  = "f1-micro"
  zone          = "europe-west1-b"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
      size  = 30
    }
  }
}

network_interface {
  network = google_compute_network.vpc.name
  access_config {
    nat_ip = google_compute_address.static.address
  }
}
```

Docker Compose

```
services:
  db:
    image: postgres:16
    volumes:
      - db_data:/var/lib/postgresql/data

  mongodb:
    image: mongo:8.0
    volumes:
      - mongo_data:/data/db

  frontend:
    build: .
    depends_on: [db, mongodb]
    environment:
      - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}

  proxy:
    build: ./nginx
    ports: ["80:80", "443:443"]
    volumes:
      - letsencrypt:/etc/letsencrypt
```

PaaS Implementation

Cloud Run Service

```
gcloud run deploy cloudappdev \
  --image gcr.io/$PROJECT/app \
  --platform managed \
  --region europe-west1 \
  --allow-unauthenticated \
  --min-instances 0 \
  --max-instances 10 \
  --memory 512Mi \
  --cpu 1
```

Terraform Managed Services

```
resource "google_sql_database_instance" "db" {
  database_version = "POSTGRES_16"
  settings {
    tier = "db-g1-small"
    edition = "ENTERPRISE"
    backup_configuration {
      enabled = true
    }
  }
}

resource "google_firestore_database" "nosql" {
  type = "FIRESTORE_NATIVE"
  database_edition = "ENTERPRISE"
}

resource "google_storage_bucket" "images" {
  location = "EU"
  uniform_bucket_level_access = true
}
```


Load Testing

Locust Framework

Periodic Workload

```
100 concurrent users  
10 Minuten  
Registration, Browse, CRUD
```

Once-in-a-Lifetime

```
1,000 concurrent users  
3 Minuten  
200 users/sec spawn rate
```

Test Coverage

- API Endpoints (CRUD)
- Database Operations
- Likes & Comments System
- Search & Pagination

Bewusst nicht getestet

- File Uploads (bandbreitenintensiv)
- Authentication (Firebase managed)

Infrastructure as Code

Deployment Automation

```
# PowerShell / Bash Scripts
./deploy.ps1 deploy    # Full deployment
./deploy.ps1 update    # Update container
./deploy.ps1 logs      # View logs
./deploy.ps1 status    # Check health
```

Automatisierte Schritte:

1. Cloud SQL Setup & Migration
2. Firestore Database Creation
3. Container Build & Push (Artifact Registry)
4. Cloud Run Deployment & Environment Configuration

Terraform Resources

```
# IaaS Resources
- google_compute_instance
- google_compute_network
- google_compute_firewall
- google_compute_address

# PaaS Resources
- google_cloud_run_service
- google_sql_database_instance
- google_firestore_database
- google_storage_bucket
- google_secret_manager_secret
```

State Management:

- Remote Backend (GCS)
- Terraform Lock

Milestone 1 - Technical Requirements

Standard Cloud Platform

Google Cloud Platform

IaaS Terraform Deployment

Compute Engine + Docker Compose

PaaS Terraform Deployment

Cloud Run + Managed Services

Performance Testing

Locust Scripts + Test Report (IaaS & PaaS)

Multi-User Authentication

Firebase Identity Server

Vielen Dank!

CloudAppDev Milestone 1

Next.js 15 + React 19 | PostgreSQL 16 + MongoDB 8.0
Google Cloud Platform (IaaS + PaaS) | Locust Load Testing